

Zde je detailně zpracovaný a doplněný studijní materiál k okruhu "20. Principy vrstvené architektury počítačových sítí, referenční model OSI. Charakteristika lokálních počítačových sítí."

20. Principy vrstvené architektury počítačových sítí, referenční model OSI. Charakteristika lokálních počítačových sítí.

Klíčové pojmy

Vrstvená architektura

Protokol

Služba

Rozhraní (Interface)

Zapouzdření (Encapsulation)

Odpouzďření (Decapsulation)

PDU (Protocol Data Unit)

Referenční model OSI

Fyzická vrstva

Linková vrstva

Síťová vrstva

Transportní vrstva

Relační vrstva

Prezentační vrstva

Aplikační vrstva

Model TCP/IP

Lokální počítačová síť (LAN)

MAC adresa

IP adresa

Port

Topologie sítě

Switch (přepínač)

Router (směrovač)

A. Principy vrstvené architektury

Síťová komunikace mezi dvěma počítači je extrémně komplexní proces, který zahrnuje mnoho aspektů: hardware, kódování signálů, směrování v globální síti, zabezpečení, operační systémy a samotné programy. Aby bylo možné takto složitý systém navrhnout, implementovat a spravovat, využívá se **vrstvená architektura**.

Hlavní principy a výhody vrstvené architektury:

Dekompozice problému: Složitý úkol se rozdělí do několika menších, jednodušších a relativně nezávislých vrstev. Každá vrstva má specifickou sadu funkcí a odpovědností.

Abstrakce a zapouzdření: Každá vrstva poskytuje své služby vrstvě bezprostředně vyšší a pro svou činnost využívá služeb vrstvy bezprostředně nižší. Detaily interní implementace dané vrstvy jsou pro ostatní vrstvy skryté (zapouzdřené). Vrstvy komunikují pouze přes definovaná rozhraní.

Standardizace rozhraní: Definovaná rozhraní mezi vrstvami umožňují nezávislý vývoj a inovace. Pokud se změní technologie uvnitř jedné vrstvy (např. přejde se z metalického kabelu na optické vlákno na nejnižší vrstvě), na vyšší vrstvy (např. webový prohlížeč) to nemá žádný vliv, protože rozhraní mezi vrstvami zůstalo stejné. To zvyšuje flexibilitu a modularitu systému.

Peer-to-peer komunikace: Virtuálně spolu komunikují vždy stejné vrstvy protilehlých systémů (např. síťová vrstva odesílatele komunikuje se síťovou vrstvou příjemce) pomocí pravidel zvaných **protokoly**. Protokol definuje formát zpráv a pravidla pro jejich výměnu.

Zjednodušení údržby a ladění: Díky rozdělení na vrstvy je snazší identifikovat a řešit problémy, protože problém lze obvykle izolovat do konkrétní vrstvy.

Zapouzdření dat (Encapsulation) při průchodu vrstvami:

Když aplikace odešle data, ta postupují shora dolů přes jednotlivé vrstvy síťového zásobníku. Každá vrstva k obdrženým datům přidá svou vlastní řídicí informaci – **hlavičku (Header)**, případně **patičku (Trailer)**. Tomuto procesu se říká **zapouzdření**. Data s přidanou hlavičkou/patičkou tvoří **Protocol Data Unit (PDU)** dané vrstvy.

Na přijímací straně proces probíhá obráceně (data jdou zdola nahoru) a každá vrstva svou hlavičku (a případně patičku) odřízne a zpracuje. Tomuto procesu se říká **odpouzdření (Decapsulation)**.

Příklad zapouzdření: 1. **Aplikační data:** DATA 2. **Transportní vrstva:** Přidá TCP hlavičku: [TCP_H] DATA (Segment) 3. **Síťová vrstva:** Přidá IP hlavičku: [IP_H] [TCP_H] DATA (Paket) 4. **Linková vrstva:** Přidá Ethernet hlavičku a patičku: [ETH_H] [IP_H] [TCP_H] DATA [ETH_T] (Rámec) 5. **Fyzická vrstva:** Převéde rámec na bity a odešle.

B. Referenční model OSI (Open Systems Interconnection)

Model OSI je teoretický, sedmivrstvý model vyvinutý organizací ISO (International Organization for Standardization) pro standardizaci síťových protokolů. Ačkoliv se v reálném Internetu používá jednodušší čtyřvrstvý model TCP/IP, model OSI zůstává standardem pro výuku a popis síťových funkcí díky své komplexnosti a logické struktuře.

Vrstvy modelu OSI (od nejvyšší po nejnižší):

7. Aplikační vrstva (Application Layer)

Účel: Poskytuje síťové služby přímo uživatelským aplikacím. Je to rozhraní mezi aplikací a síťovými službami.

Funkce: Identifikace komunikačních partnerů, určení dostupnosti prostředků, synchronizace komunikace.

Protokoly: HTTP (Hypertext Transfer Protocol), FTP (File Transfer Protocol), SMTP (Simple Mail Transfer Protocol), POP3 (Post Office Protocol 3), IMAP (Internet Message Access Protocol), DNS (Domain Name System), Telnet.

Datová jednotka (PDU): Data.

6. Prezentační vrstva (Presentation Layer)

Účel: Zajišťuje, že data jsou pro aplikaci čitelná z hlediska formátu. Převádí data do společného formátu nebo formátu srozumitelného pro cílovou aplikaci.

Funkce: Kódování znaků (např. ASCII, EBCDIC, UTF-8), komprese dat pro efektivnější přenos, šifrování a dešifrování dat (např. SSL/TLS).

Protokoly: JPEG, MPEG, ASCII, EBCDIC, TLS/SSL.

Datová jednotka (PDU): Data.

5. Relační vrstva (Session Layer)

Účel: Správa, navazování, udržování a ukončování dialogů (relací) mezi aplikacemi na koncových systémech.

Funkce: Řeší např. synchronizaci komunikace (checkpointing), správu tokenů a obnovu spojení po výpadku. Umožňuje plně duplexní, poloduplexní nebo simplexní komunikaci.

Protokoly: NetBIOS, RPC (Remote Procedure Call), PPTP (Point-to-Point Tunneling Protocol).

Datová jednotka (PDU): Data.

4. Transportní vrstva (Transport Layer)

Účel: Zajišťuje transparentní přenos dat mezi koncovými aplikacemi (End-to-End) na různých hostitelích.

Funkce: Rozděluje data z aplikací do menších celků (segmentů/datagramů), řeší adresování konkrétních programů pomocí **portů**. Může poskytovat spolehlivý (s kontrolou chyb, potvrzováním a opakovaným vysíláním) nebo nespolehlivý přenos.

Protokoly: TCP (Transmission Control Protocol – spolehlivý, orientovaný na spojení), UDP (User Datagram Protocol – nespolehlivý, bezspojoyý).

Datová jednotka (PDU): Segment (u TCP) / Datagram (u UDP).

3. Síťová vrstva (Network Layer)

Účel: Zajišťuje doručení dat mezi dvěma uzly v různých sítích (inter-network delivery).

Funkce: Stará se o logické adresování uzlů v globální síti pomocí **IP adres** a o **směrování (Routing)** – hledání optimální cesty skrze síťové prvky (routery).

Protokoly: IP (Internet Protocol – IPv4, IPv6), ICMP (Internet Control Message Protocol), OSPF (Open Shortest Path First), BGP (Border Gateway Protocol).

Zařízení: Router (směrovač).

Datová jednotka (PDU): Paket.

2. Linková vrstva (Data Link Layer)

Účel: Zajišťuje spolehlivý přenos dat v rámci jedné lokální sítě (LAN) mezi přímo sousedícími uzly (intra-network delivery).

Funkce: Řeší fyzické adresování pomocí **MAC adres (Media Access Control)**, řízení přístupu ke sdílenému přenosovému médiumu (např. CSMA/CD pro Ethernet, CSMA/CA pro Wi-Fi), detekci a případnou opravu chyb v rámci linky.

Protokoly: Ethernet (IEEE 802.3), Wi-Fi (IEEE 802.11), PPP (Point-to-Point Protocol), HDLC (High-Level Data Link Control).

Zařízení: Switch (přepínač), Bridge (most), síťová karta (NIC).

Datová jednotka (PDU): Rámec (Frame).

1. Fyzická vrstva (Physical Layer)

Účel: Specifikuje fyzické, elektrické, optické a funkční vlastnosti přenosového média.

Funkce: Převádí rámce z linkové vrstvy na surové posloupnosti bitů a moduluje je do fyzických signálů (elektrické impulzy, světelné pulzy, rádiové vlny) pro přenos po médiumu. Definuje napětí, frekvence, typy konektorů a kabeláže.

Média: Kroucená dvojlinka (UTP/STP), optický kabel, koaxiální kabel, rádiový prostor.

Zařízení: Hub (rozbočovač), repeater (opakovač), síťová karta (NIC), modemy.

Datová jednotka (PDU): Bit.

21. Technologie Ethernet, její principy a vývoj, algoritmus CSMA/CD. Bezdrátové lokální sítě standardu IEEE 802.11.

Klíčové pojmy

Ethernet: Dominantní technologie pro drátové lokální sítě (LAN), standardizovaná jako IEEE 802.3.

MAC adresa: Fyzická adresa síťového adaptéru, unikátní 48bitový identifikátor.

Rámec: Základní jednotka dat přenášená na linkové vrstvě Ethernetu.

CSMA/CD (Carrier Sense Multiple Access with Collision Detection): Přístupový mechanismus pro sdílená média v klasickém Ethernetu, detekuje a řeší kolize.

Wi-Fi (IEEE 802.11): Standard pro bezdrátové lokální sítě (WLAN).

CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance): Přístupový mechanismus pro bezdrátové sítě, snaží se předcházet kolizím.

Access Point (AP): Zařízení v bezdrátové síti, které umožňuje bezdrátovým zařízením připojit se k drátové síti.

Half-Duplex: Komunikační režim, kdy zařízení může buď vysílat, nebo přijímat, ale ne současně.

Full-Duplex: Komunikační režim, kdy zařízení může současně vysílat i přijímat.

Kolizní doména: Segment sítě, kde může dojít ke kolizím paketů.

A. Technologie Ethernet a její vývoj

Ethernet (standardizovaný jako IEEE 802.3) je dominantní technologie pro drátové lokální sítě (LAN). Definuje vlastnosti fyzické a linkové vrstvy (vrstvy 1 a 2) OSI modelu. Jeho hlavní funkcí je umožnit zařízením v lokální síti sdílet komunikační médium a efektivně přenášet data.

MAC adresa: Každé síťové rozhraní (např. síťová karta) v Ethernetové síti má unikátní 48bitovou fyzickou adresu, známou jako MAC (Media Access Control) adresa. Tato adresa je vypálena do hardwaru a slouží k identifikaci odesílatele a příjemce rámce v rámci lokální sítě.

Rámec: Data jsou v Ethernetu přenášena v jednotkách zvaných rámce. Rámec obsahuje hlavičku (s MAC adresami odesílatele a příjemce, typem protokolu), samotná data a zápatí (pro kontrolu chyb).

Vývoj Ethernetu:

Klasický (sdílený) Ethernet (např. 10BASE5, 10BASE2, 10BASE-T s hubem):

Starší verze využívaly koaxiální kabel nebo Hub (rozbočovač).

Všechna zařízení byla připojena do jednoho sdíleného média, což znamenalo, že celá síť tvořila jednu velkou kolizní doménu.

Pokud začalo vysílat více počítačů naráz, docházelo ke kolizím.

Síť fungovala v režimu **Half-Duplex** (v jeden okamžik lze buď jen vysílat, nebo jen přijímat).

Pro řízení přístupu ke sdílenému médiu a řešení kolizí se používal algoritmus **CSMA/CD**.

Moderní (přepínaný) Ethernet (např. 100BASE-TX, 1000BASE-T, 10GBASE-R):

Dnešní sítě využívají kroucenou dvojlinku (UTP/STP kabely s konektory RJ-45) nebo optická vlákna zapojená do **Switche (přepínače)**.

Switch je inteligentní zařízení, které izoluje provoz mezi jednotlivými porty. Každý port switchu představuje samostatnou kolizní doménu.

Komunikace probíhá v režimu **Full-Duplex** (současné vysílání i příjem), což eliminuje možnost kolizí na daném segmentu.

V moderním přepínaném Ethernetu ke kolizím již nedochází, a proto algoritmus CSMA/CD není aktivně využíván, ačkoliv je stále součástí standardu pro zpětnou kompatibilitu s half-duplex provozem.

B. Algoritmus CSMA/CD (Carrier Sense Multiple Access with Collision Detection)

Algoritmus CSMA/CD byl klíčový pro fungování klasického sdíleného Ethernetu, kde více stanic sdílelo jedno fyzické médium. Jeho účelem bylo zajistit, aby počítače na jednom kabelu dokázaly komunikovat, aniž by si signály navzájem zničily.

Jak algoritmus funguje (krok za krokem):

CS (Carrier Sense - Naslouchání nosné):

Stanice, která chce vysílat, nejprve "naslouchá" médiu (měří napětí na kabelu), aby zjistila, zda je volné.

Pokud je médium obsazené (jiná stanice vysílá), čeká náhodnou dobu a poté se pokusí znovu.

Pokud je volné, začne vysílat data.

MA (Multiple Access - Vícenásobný přístup):

Do sítě je připojeno více stanic současně a všechny mají stejné právo k přístupu na médium, pokud je volné.

CD (Collision Detection - Detekce kolize):

Během vysílání stanice neustále kontroluje napětí na kabelu (monitoruje médium).

Pokud dvě stanice začnou vysílat téměř ve stejný okamžik (protože obě zjistily, že médium je volné), jejich signály se protnou a vznikne **kolize** (zkreslení napětí, zvýšení úrovně signálu).

Řešení kolize:

Jam Signal: Jakmile stanice detekuje kolizi, okamžitě ukončí vysílání běžných dat a do sítě vyšle speciální varovný signál zvaný **JAM signal**. Tím zajistí, že o kolizi se dozví i všechny ostatní stanice v kolizní doméně.

Exponential Backoff: Každá stanice, která detekovala kolizi, si vygeneruje náhodný časový interval (tzv. "backoff time"). Tento interval se vypočítá pomocí algoritmu **Exponential Backoff**, který zvyšuje maximální možnou dobu čekání s každou opakovanou kolizí.

Opakování vysílání: Po uplynutí tohoto náhodného času se stanice pokusí o vysílání znovu od prvního kroku (naslouchání nosné). Náhodnost zajišťuje, že se stanice netrefí do stejného okamžiku znovu a snižuje pravděpodobnost opakované kolize.

C. Bezdrátové sítě IEEE 802.11 (Wi-Fi)

Standard IEEE 802.11 definuje pravidla pro bezdrátové lokální sítě (WLAN), komerčně známé jako Wi-Fi. Protože rádiový prostor je z podstaty sdílené médium a stanice pracují v režimu **Half-Duplex**, potýkají se s odlišnými problémy než drátové sítě, zejména s efektivní detekcí kolizí.

Problém skryté stanice (Hidden Node Problem)

U bezdrátových sítí nelze použít detekci kolizí (CD) efektivně, protože stanice nemusí slyšet všechny ostatní stanice v síti. Příklad:

- Stanice *A* vidí přístupový bod (AP).
- Stanice *C* také vidí AP.
- Nicméně *A* a *C* jsou od sebe příliš daleko a navzájem se neslyší (jsou pro sebe "skryté").
- Pokud obě stanice naslouchají, usoudí, že vzduch je čistý, začnou vysílat k AP a u přístupového bodu

dojde ke kolizi.

- Stanice *A* ani *C* se však o této kolizi nedozví, protože samy u sebe žádné cizí rušení nenaměří.

Algoritmus CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance)

Jelikož Wi-Fi neumí kolize efektivně detekovat, snaží se jim předcházet (Collision Avoidance).

Naslouchání kanálu: Stanice naslouchá, zda je rádiový kanál volný.

Čekání a náhodný odstup:

Pokud je kanál volný, nezačne vysílat ihned, ale počká povinnou dobu (tzv. **IFS - Inter-Frame Space**).

Existují různé typy IFS (např. SIFS, DIFS) pro různé typy rámců a priorit.

Následně počká ještě náhodnou dobu zvanou **Contention Window**.

Pokud je i poté kanál volný, odešle data.

Potvrzování (ACK):

Každý úspěšně doručený datový rámec musí příjemce potvrdit zasláním krátkého rámce **ACK (Acknowledge)** zpět odesílateli.

Pokud odesílatel ACK neobdrží v určitém časovém limitu, předpokládá, že došlo ke kolizi (nebo ztrátě rámce), a zkusí rámec poslat znovu (opět s využitím Exponential Backoff).

Volitelný mechanismus RTS/CTS (Request to Send / Clear to Send)

Tento mechanismus je navržen k řešení problému skryté stanice a je obvykle používán pro přenos velkých datových rámců nebo v sítích s vysokou hustotou.

RTS (Request to Send): Před odesláním samotných velkých dat může vysílající stanice poslat krátký dotaz RTS přístupovému bodu.

CTS (Clear to Send): Přístupový bod odpoví rámcem CTS.

Rezervace média: Rámec CTS slyší všechny stanice v dosahu přístupového bodu (včetně těch skrytých pro původního odesílatele) a ten jim explicitně přikáže: "Teď všichni ztichněte, bude vysílat stanice A po dobu X." Tím se efektivně rezervuje médium pro vysílající stanici a zabráni se kolizím.

Přehled základních generací Wi-Fi (standardy IEEE 802.11):

802.11a: Pásmo 5 GHz, rychlost do 54 Mb/s.

802.11b: Pásmo 2.4 GHz, rychlost do 11 Mb/s.

802.11g: Pásmo 2.4 GHz, rychlost do 54 Mb/s. Zpětně kompatibilní s 802.11b.

802.11n (Wi-Fi 4): Pásmo 2.4 GHz i 5 GHz, zavedení technologie **MIMO (Multiple-Input Multiple-Output)** pro vyšší rychlosti a dosah, širší kanály (40 MHz). Rychlosti v řádu stovek Mb/s.

802.11ac (Wi-Fi 5): Pouze pásmo 5 GHz, další vylepšení MIMO (MU-MIMO), širší kanály (až 160 MHz). Vyšší rychlosti v řádu stovek Mb/s až Gb/s.

802.11ax (Wi-Fi 6): Pásmo 2.4 GHz i 5 GHz (a nově 6 GHz pro Wi-Fi 6E), vyšší efektivita ve zahuštěných prostorech díky technologii **OFDMA (Orthogonal Frequency-Division Multiple Access)** a vylepšené MU-MIMO. Zaměřeno na zlepšení výkonu v prostředí s mnoha zařízeními.

Praktické použití

LAN síť: Základní technologie pro drátové sítě v domácnostech, kancelářích a datových centrech.

Připojení klientů: Umožňuje koncovým zařízením (počítače, telefony, tablety) připojení k síti.

Průmyslové sítě: Použití Ethernetu v průmyslové automatizaci (Industrial Ethernet).

Domácí Wi-Fi: Bezdrátové připojení k internetu a lokální síti v domácnostech.

Veřejné Wi-Fi hotspoty: Poskytování bezdrátového přístupu k internetu na veřejných místech.

Nejčastější chyby u zkoušky

Tvrzení, že moderní switchovaný Ethernet běžně používá kolize: V moderních sítích s přepínači (switches) a full-duplex komunikací k fyzickým kolizím na médiu nedochází. CSMA/CD je relevantní pouze pro half-duplex provoz na sdíleném médiu (např. s hubem).

Zaměňování MAC a IP adresy: MAC adresa je fyzická adresa síťového adaptéru na linkové vrstvě (vrstva 2), zatímco IP adresa je logická adresa zařízení na síťové vrstvě (vrstva 3), která se používá pro směrování dat mezi sítěmi.

Ignorování sdíleného média u Wi-Fi: Bezdrátové médium je vždy sdílené, což je základní důvod pro použití CSMA/CA a mechanismů jako RTS/CTS, protože detekce kolizí je obtížná a problém skryté stanice je inherentní.

Typické zkouškové otázky

Popište Ethernetový rámec.

Odpověď: Ethernetový rámec je základní jednotka dat přenášená na linkové vrstvě (vrstva 2 OSI modelu). Skládá se z několika částí:

Preamble a Start Frame Delimiter (SFD): 8 bytů pro synchronizaci přijímače.

Cílová MAC adresa: 6 bytů, identifikuje příjemce rámce.

Zdrojová MAC adresa: 6 bytů, identifikuje odesílatele rámce.

Typ/Délka: 2 byty, určuje typ protokolu vyšší vrstvy (např. IP) nebo délku dat.

Data (Payload): 46 až 1500 bytů, samotná přenášená data.

Frame Check Sequence (FCS): 4 byty, kontrolní součet pro detekci chyb v rámci.

Jak funguje CSMA/CD?

Odpověď: CSMA/CD (Carrier Sense Multiple Access with Collision Detection) je přístupový mechanismus používaný v klasickém sdíleném Ethernetu pro řízení přístupu k médiu a řešení kolizí. Funguje následovně:

Carrier Sense (Naslouchání nosné): Stanice, která chce vysílat, nejprve naslouchá médiu, aby zjistila, zda je volné. Pokud je obsazené, čeká.

Multiple Access (Vícenásobný přístup): Více stanic má stejné právo k přístupu na médium, pokud je volné.

Collision Detection (Detekce kolize): Pokud dvě stanice začnou vysílat téměř současně, dojde ke kolizi. Vysílající stanice detekuje tuto kolizi monitorováním média.

Řešení kolize: Po detekci kolize stanice vyše **JAM signal**, aby informovala ostatní stanice. Poté se obě stanice odmlčí a každá si vygeneruje náhodný časový interval (pomocí **Exponential Backoff**), po jehož uplynutí se pokusí o vysílání znovu.

Proč Wi-Fi používá CSMA/CA?

Odpověď: Wi-Fi používá CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance), protože efektivní detekce kolizí (jako u CSMA/CD) je v bezdrátovém prostředí obtížná a nespolehlivá. Hlavní důvody jsou:

Problém skryté stanice: Stanice nemusí slyšet všechny ostatní stanice v síti, a tak nemůže detekovat kolize, které se dějí mimo její dosah (např. u přístupového bodu).

Half-Duplex provoz: Bezdrátové adaptéry obvykle nemohou současně vysílat a přijímat na stejném kanálu, což znemožňuje detekci kolize během vlastního vysílání.

Místo detekce se CSMA/CA snaží kolizím předcházet pomocí mechanismů jako je naslouchání kanálu, náhodné čekání (Inter-Frame Space a Contention Window) před vysíláním a povinné potvrzování (ACK) úspěšného doručení rámců. Volitelný mechanismus RTS/CTS dále pomáhá řešit problém skryté stanice explicitní rezervací média.

Shrnutí kapitoly

Ethernet a Wi-Fi jsou klíčové technologie pro lokální sítě, které se liší médiem a způsobem přístupu ke sdílenému kanálu. Ethernet (IEEE 802.3) je drátová technologie, která v moderních přepínaných sítích eliminuje kolize a pracuje ve full-duplex režimu. V klasickém sdíleném Ethernetu se pro řízení přístupu a řešení kolizí používal algoritmus CSMA/CD. Wi-Fi (IEEE 802.11) je bezdrátová technologie, která kvůli inherentním vlastnostem rádiového média (sdílené médium, half-duplex, problém skryté stanice) používá algoritmus CSMA/CA, který se snaží kolizím předcházet a spoléhá na potvrzování doručení.

Níže je připravený studijní materiál k okruhu SZZ "Základní principy činnosti protokolů sítě Internet - IP, TCP, UDP. Domain Name System, jeho role a činnost, DNS servery, postup řešení dotazu, reverzní DNS."

22. Základní principy činnosti protokolů sítě

Internet

Klíčové pojmy

IP (Internet Protocol): Protokol síťové vrstvy pro adresaci a směrování paketů.

IP adresa: Unikátní číselný identifikátor zařízení v síti (IPv4, IPv6).

Paket: Základní jednotka dat přenášená protokolem IP.

Směrování (Routing): Proces určování cesty pro datové pakety přes síť.

MTU (Maximum Transmission Unit): Maximální velikost datového rámce, který může být přenesen přes síťové rozhraní.

TTL (Time To Live): Hodnota v IP hlavičce, která omezuje životnost paketu v síti.

TCP (Transmission Control Protocol): Spolehlivý, spojovaný transportní protokol.

UDP (User Datagram Protocol): Nespolehlivý, bezspojový transportní protokol.

Port: Číslo identifikující konkrétní aplikaci nebo službu na hostiteli.

Three-Way Handshake: Mechanismus pro navázání spojení u TCP.

Sekvenční číslo: Číslo segmentu u TCP pro zajištění správného pořadí.

Potvrzování (ACK): Mechanismus pro potvrzení příjmu dat u TCP.

DNS (Domain Name System): Distribuovaná databáze pro překlad doménových jmen na IP adresy.

Doménové jméno: Lidsky čitelná adresa (např. `tu1.cz`).

Kořenový server (Root Server): Servery na vrcholu hierarchie DNS.

TLD server (Top-Level Domain Server): Servery spravující domény nejvyššího řádu (např. `.cz`, `.com`).

Autoritativní server: Server, který drží definitivní záznamy pro konkrétní doménu.

DNS Resolver: Klient, který provádí DNS dotazy.

Rekurzivní dotaz: Dotaz, u kterého se očekává kompletní odpověď od serveru.

Iterativní dotaz: Dotaz, u kterého server odpoví nejlepší známou informací (např. odkazem na další server).

Cache: Dočasné úložiště DNS záznamů.

TTL (Time to Live): Doba platnosti DNS záznamu v cache.

Reverzní DNS (Reverse DNS Lookup): Překlad IP adresy na doménové jméno.

PTR záznam: Typ DNS záznamu používaný pro reverzní DNS.

in-addr.arpa / ip6.arpa: Speciální domény pro reverzní DNS.

A. Protokol IP (Internet Protocol)

Protokol IP operuje na **síťové vrstvě (3. vrstva OSI)**. Jeho hlavním úkolem je nespolehlivé a bezspojové doručování datových paketů od odesílatele k příjemci skrze uzly globální sítě (směrovače / routery).

Bezspojový (Connectionless): Před odesláním dat se nenavazuje žádné spojení. Každý paket je nezávislá jednotka a může putovat sítí jinou trasou. Směrovače nemají žádné informace o předchozích paketech nebo celkovém toku dat.

Nespolehlivý (Best-Effort): IP protokol negarantuje, že paket skutečně dorazí, že dorazí ve správném pořadí, nebo že nedorazí duplicitně. Pokud dojde k chybě (např. přetížení routeru, poškození dat), paket se jednoduše zahodí. Řešení spolehlivosti se nechává na vyšších vrstvách (např. TCP).

Hlavní funkce

Logické adresování: Každé síťové rozhraní má v síti svou unikátní IP adresu.

IPv4: Používá 32bitové adresy, obvykle zapisované jako čtyři desetinná čísla oddělená tečkami (např. 192.168.1.1). Poskytuje přibližně 4.3×10^9 unikátních adres.

IPv6: Používá 128bitové adresy, zapisované hexadecimálně (např.

2001:0db8:85a3:0000:0000:8a2e:0370:7334). Poskytuje obrovský adresní prostor (3.4×10^{38} unikátních adres) a zjednodušenou hlavičku.

Směrování (Routing): Routery podle cílové IP adresy v hlavičce paketu a svých směrovacích tabulek rozhodují, kterému dalšímu sousedovi (next hop) paket předají.

Fragmentace: Pokud je IP paket větší než maximální povolená velikost rámce nižší vrstvy (MTU - Maximum Transmission Unit), IP protokol ho dokáže rozdělit na menší kusy (fragментy). Tyto fragментy jsou na cílové stanici opět složeny. V IPv6 je fragmentace na mezilehlých routerech zakázána, fragmentaci musí provést odesílatel.

Struktura IP hlavičky (zjednodušeně)

IP hlavička obsahuje klíčové informace pro směrování a zpracování paketu. Mezi nejdůležitější pole patří:

- **Verze:** Určuje verzi IP protokolu (IPv4 nebo IPv6).
- **Délka hlavičky:** Délka IP hlavičky.
- **Délka paketu:** Celková délka IP paketu (hlavička + data).
- **Time To Live (TTL):** Počet skoků (hopů), které paket může vykonat. Každý router sníží TTL o 1. Pokud TTL dosáhne 0, paket je zahozen, což zabraňuje nekonečnému putování paketů v síti.
- **Protokol:** Určuje protokol vyšší vrstvy, který je zapouzdřen v datové části IP paketu (např. 6 pro TCP, 17 pro UDP).
- **Kontrolní součet hlavičky (Header Checksum):** Pro IPv4, slouží k detekci chyb v hlavičce. V IPv6 byl odstraněn.
- **Zdrojová IP adresa:** IP adresa odesílatele.
- **Cílová IP adresa:** IP adresa příjemce.

Praktické použití

Základ veškeré internetové komunikace.

Směrování dat v globálních i lokálních sítích.

Zajištění konektivity mezi různými sítěmi.

Nejčastější chyby u zkoušky

Zaměňování IP adresy (logická adresa síťové vrstvy) a MAC adresy (fyzická adresa linkové vrstvy).

Neznalost bezspojového a nespolehlivého charakteru IP protokolu.

Nepochopení role TTL.

Paměťová pomůcka

IP je jako "pošták", který doručí dopis (paket) na správnou adresu (IP adresa), ale negarantuje, že dopis dorazí, že dorazí v pořádku nebo ve správném pořadí.

Typické zkouškové otázky

Co je hlavním úkolem protokolu IP a na jaké vrstvě OSI modelu operuje?

Odpověď: Hlavním úkolem protokolu IP je nespolehlivé a bezspojové doručování datových paketů mezi koncovými uzly v síti. Operuje na síťové vrstvě (3. vrstva) OSI modelu.

Vysvětlete pojmy "bezspojový" a "nespolehlivý" v kontextu IP.

Odpověď:

Bezspojoivý znamená, že před odesláním dat se nenavazuje žádné spojení a každý paket je zpracováván nezávisle. Pakety mohou putovat různými cestami.

Nespolehlivý (best-effort) znamená, že IP protokol negarantuje doručení paketu, správné pořadí, ani detekci či opravu chyb v datové části. Ztracené nebo poškozené pakety se jednoduše zahodí.

Jaký je rozdíl mezi IPv4 a IPv6 adresami?

Odpověď: IPv4 adresy jsou 32bitové a obvykle se zapisují jako čtyři desetinná čísla oddělená tečkami (např. 192.168.1.1). IPv6 adresy jsou 128bitové a zapisují se hexadecimálně (např. 2001:db8::1). IPv6 poskytuje mnohem větší adresní prostor a má zjednodušenou hlavičku, která například odstraňuje fragmentaci na mezilehlých routerech.

B. Transportní protokoly: TCP vs. UDP

Transportní vrstva (**4. vrstva OSI**) zajišťuje komunikaci mezi konkrétními procesy (aplikacemi) na koncových zařízeních pomocí **portů**. Port je 16bitové číslo, které identifikuje službu nebo aplikaci (např. HTTP používá port 80, HTTPS 443, DNS 53). K tomu využívá dva zcela odlišné protokoly: TCP a UDP.

1. TCP (Transmission Control Protocol)

Charakteristika: Spolehlivý protokol orientovaný na spojení. Zajišťuje, že data dorazí v pořádku, kompletní a ve správném pořadí.

Mechanismy TCP

Navázání spojení (Three-Way Handshake): Před přenosem dat si klient a server vymění tři synchronizační pakety pro navázání logického spojení:

SYN (Synchronize): Klient odešle serveru segment s žádostí o navázání spojení a své počáteční sekvenční číslo.

SYN-ACK (Synchronize-Acknowledge): Server odpoví segmentem, který potvrzuje příjem SYN od klienta (ACK) a zároveň odesílá své vlastní počáteční sekvenční číslo (SYN).

ACK (Acknowledge): Klient potvrdí příjem SYN-ACK od serveru. Spojení je navázáno.

Zajištění spolehlivosti (Potvrzování - ACK): Příjemce musí každý doručенý segment potvrdit (ACK). Pokud odesílateli nedorazí potvrzení včas, odešle data znovu.

Řazení do pořadí (Sequencing): Segmenty obsahují sekvenční čísla. Pokud dorazí např. kvůli routování napřeskáčku, TCP je před předáním aplikaci poskládá do správného pořadí.

Řízení toku dat (Flow Control - Sliding Window): Příjemce dává odesílateli vědět, jak velký má buffer (tzv. "okno"), aby ho odesílatel nezahltl. Odesílatel pak posílá jen tolik dat, kolik se vejde do okna příjemce.

Řízení zahlcení sítě (Congestion Control): TCP monitoruje stav sítě. Pokud síť začne zahazovat pakety (indikace zahlcení), TCP automaticky zpomalí rychlost vysílání (např. pomocí algoritmů jako Slow Start a Congestion Avoidance), aby zabránil dalšímu zahlcení.

Ukončení spojení: Používá podobný čtyřcestný handshake s FIN a ACK segmenty.

Struktura TCP segmentu (zjednodušeně)

Zdrojový port: Číslo portu odesílající aplikace.

Cílový port: Číslo portu cílové aplikace.

Sekvenční číslo: Číslo prvního bytu dat v tomto segmentu.

Potvrzovací číslo (Acknowledgement Number): Očekávané sekvenční číslo dalšího bytu od protistrany.

Délka hlavičky: Délka TCP hlavičky.

Príznaiky (Flags): Např. SYN, ACK, FIN, RST (reset), PSH (push), URG (urgent).

Velikost okna (Window Size): Počet bytů, které je příjemce schopen přijmout.

Kontrolní součet (Checksum): Pro detekci chyb v hlavičce i datech.

Použití

Web (HTTP/HTTPS)

E-mail (SMTP, IMAP, POP3)

Přenos souborů (FTP)

Vzdálený přístup (SSH)

Všude, kde nesmí chybět ani jeden bit dat a spolehlivost je klíčová.

2. UDP (User Datagram Protocol)

Charakteristika: Nespolehlivý, bezspojový protokol s minimální režii.

Princip

UDP jednoduše vezme data z aplikace, přidá kratičkou hlavičku (obsahující pouze porty, délku a kontrolní součet) a odešle je. Nezajímá ho, zda příjemce existuje, zda data dorazila, nebo zda dorazila ve správném pořadí.

Výhody

Extrémní rychlost: Žádné zpoždění kvůli navazování spojení, potvrzování nebo čekání na ztracené pakety.

Nízká režie: Jednoduchá hlavička a minimální zpracování na obou stranách.

Struktura UDP datagramu (zjednodušeně)

Zdrojový port: Číslo portu odesílající aplikace.

Cílový port: Číslo portu cílové aplikace.

Délka: Celková délka UDP datagramu (hlavička + data).

Kontrolní součet (Checksum): Pro detekci chyb v hlavičce i datech. Je volitelný, ale obvykle se používá.

Použití

Streamování videa/audia v reálném čase (např. IPTV, videokonference).

Online hry.

VoIP (telefonování přes internet).

Dotazy na DNS.

Pokud se ztratí jeden vzorek zvuku při hovoru, je lepší ho ignorovat, než hovor pozastavit a čekat na znovuvyslání. Aplikace mohou implementovat vlastní mechanismy spolehlivosti, pokud je to nutné.

Praktické použití

TCP: Webové prohlížení, e-mailová komunikace, stahování souborů, vzdálená správa.

UDP: Živé vysílání, videokonference, online hry, DNS dotazy, SNMP (Simple Network Management Protocol).

Nejčastější chyby u zkoušky

Zaměňování spolehlivosti a bezspojovosti.

Neznalost mechanismů, kterými TCP zajišťuje spolehlivost (potvrzování, sekvenční čísla, řízení toku/zahlcení).

Nepochopení, proč se UDP používá i přes jeho nespolehlivost (nízká latence, real-time aplikace).

Paměťová pomůcka

TCP: "Telefonní hovor" – navážeš spojení, mluvíš, potvrdíš příjem, pokud něco neslyšíš, požádáš o zopakování.

UDP: "Posílání pohlednic" – pošleš a doufáš, že dorazí. Pokud se nějaká ztratí, neřešíš to, protože už je stará.

Typické zkouškové otázky

Porovnejte TCP a UDP z hlediska spolehlivosti, navazování spojení a režie.

Odpověď:

Spolehlivost: TCP je spolehlivý (garantuje doručení, pořadí, integritu), UDP je nespolehlivý (best-effort, bez garancí).

Navazování spojení: TCP je spojovaný (používá Three-Way Handshake), UDP je bezspojový (data se posílají okamžitě).

Režie: TCP má vyšší režii kvůli mechanismům spolehlivosti a řízení, UDP má minimální režii.

Popište Three-Way Handshake u TCP.

Odpověď: Three-Way Handshake je proces navazování TCP spojení. Zahrnuje tři kroky:

Klient odešle serveru segment **SYN** (Synchronize).

Server odpoví segmentem **SYN-ACK** (Synchronize-Acknowledge), který potvrzuje příjem SYN a zároveň odesílá své vlastní SYN.

Klient potvrdí příjem SYN-ACK segmentem **ACK** (Acknowledge). Po tomto kroku je spojení navázáno a data mohou být přenášena.

Uveďte příklady aplikací, které preferují TCP, a aplikací, které preferují UDP, a vysvětlete proč.

Odpověď:

TCP preferují: Web (HTTP/HTTPS), e-mail (SMTP), přenos souborů (FTP). Tyto aplikace vyžadují, aby všechna data dorazila kompletní a ve správném pořadí, protože ztráta byť jediného bitu by mohla vést k poškození souboru nebo nesprávnému zobrazení stránky.

UDP preferují: Streamování videa/audia, online hry, VoIP (telefonování přes internet), DNS dotazy. U těchto aplikací je klíčová nízká latence a rychlost. Ztráta malého množství dat (např. jednoho zvukového vzorku) je přijatelnější než zpoždění způsobené opakovaným odesíláním ztracených dat.

C. Domain Name System (DNS)

Lidé si snadno pamatují textové názvy (např. `tu1.cz`), ale počítače a routery potřebují ke komunikaci IP adresy (např. `147.230.1.2`). **DNS je celosvětová distribuovaná a hierarchická databáze**, jejímž hlavním úkolem je překlad doménových jmen na IP adresy.

Hierarchická struktura DNS

DNS netvoří jeden centrální server, ale stromová struktura serverů, která odráží hierarchii doménových jmen:

Kořenové servery (Root Servers): Stojí na vrcholu pyramidy (značí se jako tečka `.`). Existuje 13 logických kořenových serverů (označených A až M), které jsou fyzicky replikovány po celém světě. Vědí, které servery spravují konkrétní TLD (Top-Level Domains).

Servery nejvyššího řádu (TLD - Top-Level Domain Servers): Spravují domény jako `.cz`, `.com`, `.org`, `.net`, `.gov` atd. Vědí, které autoritativní servery spravují konkrétní domény druhého řádu (např. `tu1.cz`).

Autoritativní servery (Authoritative Name Servers): Jsou servery konkrétní organizace (např. univerzity nebo firmy), které drží finální, definitivní záznamy o konkrétních doménách a subdoménách (např. vědí přesnou IP adresu pro `www.tu1.cz` nebo `stag.tu1.cz`).

Local DNS Resolver (Caching Name Server): Není součástí hierarchie DNS, ale je to klíčový prvek na straně klienta (obvykle server poskytovatele internetu - ISP, nebo veřejný DNS server jako Google DNS 8.8.8.8). Přijímá rekurzivní dotazy od klientů a provádí iterativní dotazy do hierarchie DNS. Výsledky si ukládá do cache.

Postup řešení DNS dotazu (Rekurzivní vs. Iterativní)

Když do prohlížeče zadáš adresu `tu1.cz`, procesor tvého počítače (tzv. **DNS Resolver**) provede následující kroky:

Kontrola cache: Nejprve se podívá do lokální paměti počítače (hosts soubor, DNS cache operačního systému) a do cache tvého routeru. Pokud tam záznam z minula je a nevypršela jeho životnost (**TTL - Time to Live**), rovnou ho použije.

Dotaz na Local DNS Server: Pokud v cache nic není, počítač pošle **rekurzivní dotaz** na svůj nakonfigurovaný lokální DNS server (např. server ISP nebo Google DNS). Rekurzivní dotaz znamená: "Já nic nevím, ty mi sežeň výsledek a rovnou mi ho přines."

Iterativní kolečko (provádí Local DNS Server): Lokální DNS server, který obdržel rekurzivní dotaz, nyní sám provádí sérii **iterativních dotazů** do hierarchie DNS:

Local DNS se zeptá **Kořenového serveru** (.). Ten odpoví: "IP adresu pro `tu1.cz` neznám, ale tady máš IP adresu TLD serveru pro `.cz`."

Local DNS se zeptá **TLD serveru pro .cz**. Ten odpoví: "IP adresu pro `tu1.cz` neznám, ale tady máš IP adresu autoritativního serveru, který spravuje přímo doménu `tu1.cz`."

Local DNS se zeptá **Autoritativního serveru TUL**. Tento server má záznamy přímo ve své databázi, takže odpoví: "Ano, doména `tu1.cz` má IP adresu `147.230.1.2`."

Odpověď klientovi: Lokální DNS server si tento výsledek uloží do své cache (pro příště, po dobu TTL) a předá ho tvému počítači. Teprve teď může prohlížeč začít navazovat TCP spojení se serverem

`147.230.1.2`.

Typy DNS záznamů (Resource Records)

DNS servery ukládají různé typy záznamů, které definují, jak se doménová jména mapují na jiné zdroje:

A (Address Record): Překlad doménového jména na **IPv4** adresu.

Příklad: `tu1.cz IN A 147.230.1.2`

AAAA (IPv6 Address Record): Překlad doménového jména na **IPv6** adresu.

Příklad: `tu1.cz IN AAAA 2001:db8::1`

CNAME (Canonical Name Record): Alias – říká, že jedna doména je jen ukazatelem na jinou doménu (kanonické jméno).

Příklad: `www.tu1.cz IN CNAME tu1.cz` (`www.tu1.cz` je alias pro `tu1.cz`)

MX (Mail Exchanger Record): Určuje poštovní servery, které pro danou doménu přebírají e-maily.

Obsahuje prioritu a doménové jméno poštovního serveru.

Příklad: `tu1.cz IN MX 10 mail.tu1.cz`

NS (Name Server Record): Určuje IP adresy autoritativních DNS serverů pro danou doménu.

Příklad: `tu1.cz IN NS ns1.tu1.cz`

SRV (Service Record): Určuje umístění (hostname a port) serverů pro specifické služby (např. SIP, XMPP, LDAP).

Příklad: `_sip._tcp.tu1.cz IN SRV 10 0 5060 sipserver.tu1.cz`

TXT (Text Record): Umožňuje přidat libovolný textový řetězec k doméně. Často se používá pro ověřování domény, SPF (Sender Policy Framework) nebo DKIM (DomainKeys Identified Mail) pro e-mailovou bezpečnost.

Příklad: `tu1.cz IN TXT "v=spf1 include:_spf.tu1.cz ~all"`

Reverzní DNS (Reverse DNS lookup / PTR záznam)

Reverzní DNS dělá přesný opak běžného DNS dotazu – překládá známou IP adresu zpět na doménové jméno.

Princip: Využívá speciální domény:

Pro **IPv4** adresy se používá doména `in-addr.arpa`. IP adresa se zapíše v obráceném pořadí a připojí se k této doméně (např. z IP `147.230.1.2` vznikne dotaz na `2.1.230.147.in-addr.arpa`).

Pro **IPv6** adresy se používá doména `ip6.arpa`. IPv6 adresa se rozdělí na jednotlivé hexadecimální číslice, obrátí se pořadí a připojí se k `ip6.arpa`.

Hledá se tzv. **PTR (Pointer) záznam**.

Praktické využití

Ochrana proti spamu a zabezpečení: Poštovní servery (např. Gmail) si při příjmu e-mailu ověřují, zda IP adresa odesílajícího serveru odpovídá doméně, kterou se server prezentuje v e-mailové hlavičce. Pokud reverzní záznam chybí nebo nesouhlasí, e-mail je často označen jako spam nebo zcela zahozen.

Logování a auditování: V log souborech serverů se často místo IP adres ukládají doménová jména, což usnadňuje čitelnost a analýzu.

Diagnostika sítě: Nástroje jako `traceroute` nebo `netstat` mohou používat reverzní DNS pro zobrazení doménových jmen místo IP adres.

Praktické použití

Přístup k webovým stránkám a online službám.

E-mailová komunikace.

Síťová konfigurace a správa.

Zabezpečení a ověřování identity serverů.

Nejčastější chyby u zkoušky

Neznalost hierarchie DNS a role jednotlivých typů serverů.

Zaměňování rekurzivního a iterativního dotazu.

Nepochopení účelu a mechanismu reverzního DNS.

Neznalost základních typů DNS záznamů.

Paměťová pomůcka

DNS je "telefonní seznam" internetu, který převádí jména na čísla a naopak.

Typické zkouškové otázky

Vysvětlete roli a hierarchickou strukturu DNS.

Odpověď: DNS (Domain Name System) je distribuovaná a hierarchická databáze, jejímž hlavním úkolem je překlad lidsky čitelných doménových jmen na číselné IP adresy a naopak. Hierarchie se skládá z kořenových serverů (.), TLD serverů (např. `.cz`, `.com`) a autoritativních serverů (spravujících konkrétní domény jako `tul.cz`).

Popište postup řešení DNS dotazu od klienta až po autoritativní server.

Odpověď:

Klient nejprve zkontroluje svou lokální cache.

Pokud záznam nenajde, odešle rekurzivní dotaz na svůj lokální DNS server (resolver).

Lokální DNS server provede sérii iterativních dotazů: nejprve se zeptá kořenového serveru, ten ho odkáže na TLD server, TLD server ho odkáže na autoritativní server pro danou doménu.

Autoritativní server vrátí IP adresu lokálnímu DNS serveru.

Lokální DNS server si IP adresu uloží do cache a předá ji klientovi.

Co je reverzní DNS a k čemu se používá?

Odpověď: Reverzní DNS je proces překladu IP adresy zpět na doménové jméno. Využívá speciální domény `in-addr.arpa` (pro IPv4) nebo `ip6.arpa` (pro IPv6) a hledá PTR záznamy. Používá se primárně pro bezpečnostní účely (např. ověřování odesílatelů e-mailů pro boj proti spamu), logování a síťovou diagnostiku.

Vyjmenujte a popište alespoň tři typy DNS záznamů.

Odpověď:

A (Address Record): Mapuje doménové jméno na IPv4 adresu.

AAAA (IPv6 Address Record): Mapuje doménové jméno na IPv6 adresu.

CNAME (Canonical Name Record): Vytváří alias (přezdívku) pro jiné doménové jméno.

MX (Mail Exchanger Record): Určuje poštovní servery zodpovědné za příjem e-mailů pro danou doménu.

NS (Name Server Record): Určuje autoritativní DNS servery pro danou doménu.

23. Základní principy WWW, HTTP, HTML

A. Základní principy WWW (World Wide Web)

Definition: WWW je globální, distribuovaný informační systém pro sdílení a prohlížení hypertextových dokumentů a dalších zdrojů přes internet. Byl navržen Timem Bernersem-Leem v roce 1989 v CERNu.

Architektura: WWW funguje na principu klient-server a je postavena na třech základních pilířích:

URI / URL (Uniform Resource Identifier / Locator): Jednotný systém adresování, který umožňuje jednoznačně identifikovat a lokalizovat jakýkoliv zdroj (stránku, obrázek, soubor) na internetu.

Příklad: `https://tul.cz/index.html`

HTTP / HTTPS (Hypertext Transfer Protocol / Secure): Přenosový protokol, který definuje pravidla komunikace mezi klientem a serverem pro výměnu webových zdrojů.

HTML (Hypertext Markup Language): Formát (značkovací jazyk), ve kterém jsou webové dokumenty napsány, aby jim webový prohlížeč rozuměl a dokázal je správně vykreslit.

Princip Klient-Server:

Klient (Webový prohlížeč / User Agent): Software (např. Chrome, Firefox), který odešle požadavek na server, přijme zdrojová data (HTML, CSS, JavaScript) a vykreslí je uživateli do interaktivní podoby.

Server (Webový server, např. Apache, Nginx): Software, který naslouchá na standardních portech (80 pro HTTP, 443 pro HTTPS), přijímá požadavky od klientů, zpracovává je (případně komunikuje s databází nebo spouští serverové skripty jako PHP/Python) a posílá zpět odpovídající odpověď.

B. Protokol HTTP (Hypertext Transfer Protocol)

Definition: HTTP je aplikační protokol (7. vrstva OSI modelu) určený pro efektivní a spolehlivý přenos hypertextových dokumentů a dalších dat. Pracuje nad spolehlivým transportním protokolem TCP.

Hlavní charakteristiky HTTP:

Bezstavový (Stateless): Server si sám o sobě nepamatuje předchozí interakce s klientem. Každý požadavek je zpracován jako nezávislá operace. Pro udržení stavu mezi požadavky se používají mechanismy jako Cookies a Sessions (viz otázka č. 26).

Textový (HTTP/1.1): Komunikace probíhá ve formě lidsky čitelných textových řetězců.

Binární (HTTP/2, HTTP/3): Moderní verze HTTP (HTTP/2 a HTTP/3) používají interně binární formát pro efektivnější přenos a multiplexování více požadavků přes jedno TCP spojení, čímž se snižuje latence a zvyšuje propustnost. Logická struktura požadavků a odpovědí však zůstává zachována.

Struktura HTTP komunikace:

Skládá se ze dvou hlavních částí: **Požadavek (Request)** a **Odpověď (Response)**. Oba obsahují úvodní řádek, sadu hlaviček (metadata) a volitelné tělo (body) s daty.

Příklad hlaviček:

Request Headers: `User-Agent` , `Accept` , `Content-Type` , `Host` , `Cookie` .

Response Headers: `Content-Type` , `Set-Cookie` , `Cache-Control` , `Server` , `Date` .

1. HTTP Požadavek (Request):

Obsahuje **HTTP metodu**, která určuje typ operace, kterou chce klient provést na zdroji.

Běžné HTTP metody:

`GET` : Požadavek na stažení dat ze serveru. Je **idempotentní** (opakované volání má stejný efekt) a **bezpečný** (nemění stav serveru). Nemá tělo.

`POST` : Odeslání dat na server ke zpracování (např. vyplnění formulář, nahrání souboru). Data jsou obsažena v těle požadavku. Není idempotentní ani bezpečný.

PUT : Nahrazení celého zdroje daty v těle požadavku. Je idempotentní.

PATCH : Částečná aktualizace zdroje. Není idempotentní.

DELETE : Požadavek na smazání zdroje. Je idempotentní.

HEAD : Stejně jako GET, ale server vrací pouze hlavičky, nikoli tělo odpovědi. Používá se pro získání metadat.

OPTIONS : Zjištění podporovaných metod a dalších komunikačních možností pro daný zdroj.

2. HTTP Odpověď (Response):

Obsahuje **stavový kód (Status Code)**, který číselně vyjadřuje výsledek operace.

Skupiny stavových kódů:

1xx – **Informativní**: Požadavek byl přijat a zpracování pokračuje (např. `100 Continue`).

2xx – **Úspěch**: Akce byla úspěšně přijata, pochopena a přijata (např. `200 OK` , `201 Created` , `204 No Content`).

3xx – **Přesměrování**: Pro dokončení akce je nutné provést další kroky (např. `301 Moved Permanently` , `302 Found` , `304 Not Modified`).

4xx – **Chyba klienta**: Požadavek obsahuje chybnou syntaxi nebo nemůže být splněn (např. `400 Bad Request` , `401 Unauthorized` , `403 Forbidden` , `404 Not Found`).

5xx – **Chyba serveru**: Serveru se nepodařilo splnit platný požadavek (např. `500 Internal Server Error` , `502 Bad Gateway` , `503 Service Unavailable`).

REST (Representational State Transfer): HTTP metody jsou klíčové pro návrh RESTful API, kde každá metoda má specifický sémantický význam pro interakci se zdroji.

C. Jazyk HTML a (X)HTML

Definice: HTML (Hypertext Markup Language) je značkovací jazyk, který definuje strukturu a sémantiku webové stránky. Využívá k tomu systém tagů (značek) uzavřených v ostrých závorkách (např.

```
<h1>Nadpis</h1> , <p>Odstavce</p> ).
```

Deklarace typu dokumentu (<!DOCTYPE html>): Na začátku každého HTML dokumentu by měla být deklarace `<!DOCTYPE html>` , která informuje prohlížeč o verzi HTML a zajišťuje, že stránka bude vykreslena ve "standardním režimu" (standards mode), nikoli v "quirks mode".

Vývoj a charakteristika (X)HTML:

Klasické HTML (do HTML 4.01): Bylo velmi tolerantní k syntaktickým chybám. Prohlížeče se snažily "uhodnout" záměr autora a vykreslit stránku i s chybějícími ukončovacími tagy nebo porušenou strukturou. To vedlo k nekonzistentnímu zobrazení napříč prohlížeči.

XHTML (Extensible Hypertext Markup Language): Vzniklo přeformulováním HTML pomocí přísných pravidel jazyka XML.

Charakteristika XHTML:

Vyžaduje naprosto precizní a validní syntaxi.

Všechny tagy a atributy musí být psány malými písmeny.

Všechny atributy musí být v uvozovkách.

Každý tag musí být korektně uzavřen (např. `<br />` místo `<br>`).

Pokud XHTML obsahovalo syntaktickou chybu, prohlížeč správně neměl stránku vůbec vykreslit a měl zobrazit chybové hlášení XML parseru.

Důvod vzniku: Cílem bylo vytvořit robustnější a konzistentnější web, který by byl snadněji zpracovatelný stroji a kompatibilní s XML nástroji.

Moderní HTML5: XHTML se ukázalo pro reálný svět jako příliš rigidní a jeho striktní pravidla často bránila rychlému vývoji. Komunita proto vytvořila standard HTML5, který v sobě spojuje pragmatismus

klasického HTML s moderními funkcemi.

Charakteristika HTML5:

Je méně striktní než XHTML, ale stále podporuje validní a sémantický kód.

Přineslo **sémantické tagy** (např. `<header>`, `<nav>`, `<article>`, `<section>`, `<footer>`, `<aside>`), které zlepšují strukturu dokumentu, přístupnost (pro čtečky obrazovky) a SEO.

Nativní podpora pro **multimédia** (tagy `<audio>`, `<video>`).

Nová **API pro JavaScript** (např. `<canvas>` pro kreslení grafiky, Geolocation API, Web Storage API).

Zlepšená podpora pro **formuláře** (nové typy vstupů jako `email`, `date`, `number`).

Document Object Model (DOM): Když prohlížeč načte HTML dokument, vytvoří z něj stromovou strukturu nazvanou DOM. Tato struktura reprezentuje dokument jako objekty a uzly, které mohou být programově manipulovány pomocí JavaScriptu.

Možnosti jazyka HTML:

Definuje **logickou strukturu textu** (nadpisy, odstavce, seznamy, tabulky).

Umožňuje vytvářet **hypertextové odkazy** (`<a href="...">`), které propojují dokumenty napříč celým internetem.

Umožňuje vkládat **externí objekty** – obrázky (`<img>`), videa (`<video>`), audio (`<audio>`) a **formuláře** (`<form>`) pro interakci s uživatelem.

Omezení jazyka HTML:

Není to programovací jazyk: HTML nemá logiku, proměnné, podmínky ani cykly. Nedokáže samo o sobě nic vypočítat ani provádět složité operace.

Neřeší vzhled (vizuální prezentaci): HTML definuje pouze *co* na stránce je (strukturu a sémantiku), ale ne *jak* to vypadá. Stylování se kompletně deleguje na **CSS (Cascading Style Sheets)** (viz otázka č. 24).

Statičnost: Web napsaný čistě v HTML je statický. Nedokáže reagovat na akce uživatele nebo dynamicky měnit obsah bez opětovného načítání celé stránky ze serveru. Pro dynamiku na straně klienta je nutné použít **JavaScript** (viz otázka č. 25).

Separace zájmů: Klíčovým principem moderního webového vývoje je oddělení struktury (HTML), prezentace (CSS) a chování (JavaScript).

Typické zkouškové otázky

Vysvětlete základní pilíře WWW a princip klient-server architektury.

Odpověď: Základní pilíře WWW jsou URI/URL (pro identifikaci a lokalizaci zdrojů), HTTP/HTTPS (pro přenos dat) a HTML (pro strukturu dokumentů). Architektura funguje na principu klient-server, kde klient (webový prohlížeč) odesílá požadavky na server (webový server), který je zpracovává a posílá zpět odpovědi s daty, jež klient vykreslí.

Popište protokol HTTP, jeho hlavní charakteristiky a strukturu komunikace (požadavek, odpověď, metody, stavové kódy).

Odpověď: HTTP je aplikační protokol pro přenos hypertextových dokumentů. Je bezstavový (server si nepamatuje předchozí interakce) a v HTTP/1.1 textový (v HTTP/2 a HTTP/3 binární pro efektivitu). Komunikace se skládá z požadavku (Request) a odpovědi (Response). Požadavek obsahuje HTTP metodu (např. GET pro získání dat, POST pro odeslání dat, PUT/PATCH pro aktualizaci, DELETE pro smazání) a hlavičky. Odpověď obsahuje stavový kód (např. 200 OK pro úspěch, 301 pro přesměrování, 404 Not Found pro chybu klienta, 500 Internal Server Error pro chybu serveru) a hlavičky.

Porovnejte HTML, XHTML a HTML5 z hlediska jejich charakteristik, možností a omezení.

Odpověď:

HTML (do 4.01): Benevolentní k chybám, což vedlo k nekonzistentnímu vykreslování. Definovalo strukturu.

XHTML: Striktní XML varianta HTML, vyžadující precizní syntaxi (malá písmena, uvozovky pro atributy, uzavřené tagy). Cílem byla robustnost a strojové zpracování, ale bylo příliš rigidní pro rychlý vývoj.

HTML5: Moderní standard, který kombinuje pragmatismus s novými funkcemi. Přinesl sémantické tagy (`<header>` , `<nav>` , `<article>`), nativní multimédia (`<audio>` , `<video>`) a API pro JavaScript.

Možnosti HTML: Definuje logickou strukturu textu, vytváří hypertextové odkazy, umožňuje vkládat externí objekty a formuláře.

Omezení HTML: Není programovací jazyk (nemá logiku), neřeší vzhled (to je role CSS) a je statické (pro dynamiku je potřeba JavaScript).

Vysvětlete koncept bezstavovosti HTTP a jak se v praxi řeší udržení stavu.

Odpověď: Bezstavovost HTTP znamená, že každý požadavek od klienta je serverem zpracován jako zcela nezávislá operace, bez znalosti předchozích požadavků. Server si sám o sobě nepamatuje kontext interakce. Udržení stavu se v praxi řeší pomocí mechanismů jako **Cookies** (malé textové soubory ukládané v prohlížeči klienta, které serveru posílá s každým požadavkem) a **Sessions** (stav je udržován na straně serveru a klientovi je předán pouze identifikátor relace, typicky v cookie).

Jaký je rozdíl mezi sémantickými a nesémantickými HTML tagy a proč jsou sémantické tagy v HTML5 důležité?

Odpověď:

Nesémantické tagy (např. `<div>` , `<span>`) pouze definují blokovou nebo řádkovou oblast bez jakéhokoli významu pro obsah.

Sémantické tagy (např. `<header>` , `<nav>` , `<article>` , `<footer>` , `<aside>`) dodávají obsahu význam a strukturu.

Důležitost sémantických tagů v HTML5: Zlepšují **přístupnost** (pro čtečky obrazovky a asistivní technologie), **SEO** (vyhledávače lépe rozumí obsahu stránky), **čitelnost kódu** pro vývojáře a umožňují **snadnější stylování** pomocí CSS, protože poskytují jasné body pro cílení stylů.

Zde je dokonalý studijní materiál k okruhu SZZ č. 24, doplněný a naformátovaný dle vašich přísných pravidel:

24. CSS vlastnosti, hodnoty, dědění, kaskádování.

Blokový model CSS.

Klíčové pojmy

selektor

vlastnost

hodnota

kaskáda

specifita

dědění

blokový model (box model)

margin

padding

border

content

box-sizing

display (block, inline, flex)

A. CSS Vlastnosti a Hodnoty

CSS (Cascading Style Sheets) slouží k popisu vzhledu a grafického formátování dokumentů napsaných v HTML.

Struktura CSS kódu se skládá z pravidel:

Selektor: Určuje, na které HTML elementy se styl aplikuje (např. podle tagu `p`, třídy `.trida`, nebo ID `#identifikator`).

Deklarace: Blok ohraničený složenými závorkami `{ }`, který obsahuje dvojice `vlastnost: hodnota;` (property: value;).

```
p {  
  color: red;           /* Vlastnost: barva, Hodnota: červená */  
  font-size: 16px;      /* Vlastnost: velikost písma, Hodnota: 16 pixelů */  
  display: block;       /* Vlastnost: typ zobrazení, Hodnota: blokový */  
}
```

Typy hodnot:

Délkové jednotky:

Absolutní: `px` (pixels – pevná velikost).

Relativní: `em` (relativně k velikosti písma rodiče), `rem` (relativně k velikosti písma kořenového elementu `<html>`), `%` (procenta z velikosti rodičovského prvku), `vw` / `vh` (procenta z šířky/výšky okna prohlížeče).

Barvy: Klíčová slova (`red`), Hexadecimální zápis (`#FF0000`), RGB/RGBA složky (`rgba(255, 0, 0, 0.5)` – s alfa kanálem pro průhlednost).

Další typy: Klíčová slova (`auto`, `none`), URL (`url('obrazek.png')`), čísla, řetězce.

B. Dědění (Inheritance) a Kaskádování (Cascading)

Tyto dva principy řeší, jak se styly šíří hierarchií dokumentu a co se stane, když se více pravidel pokusí nastylovat stejný prvek odlišně.

1. Dědění (Inheritance)

Některé vlastnosti zadané rodičovskému elementu automaticky přecházejí (dědí se) na jeho potomky.

Co se dědí: Typicky vlastnosti týkající se textu – `font-family`, `color`, `line-height`, `text-align`, `font-size`.

Co se nedědí: Vlastnosti ovlivňující prostorové uspořádání – `margin`, `padding`, `border`, `width`, `height`, `background-color`. (Nedávalo by smysl, aby potomek automaticky přebíral například vnější okraj svého rodiče).

2. Kaskádování (Cascading)

Kaskádování je proces hledání a řešení konfliktů, kdy se na jeden prvek vztahuje více různých CSS deklarací pro stejnou vlastnost. O tom, které pravidlo vyhraje, rozhodují tři kritéria (v tomto pořadí důležitosti):

Původ a důležitost (Origin & Importance):

Styly od vývojáře mají vyšší prioritu než výchozí styly prohlížeče.

Pravidlo označené příznakem `!important` přebije téměř cokoliv jiného (nedoporučuje se používat, protože narušuje kaskádu).

Pořadí původu (od nejnižší priority):

Uživatelské styly prohlížeče (User agent stylesheets)

Uživatelské styly (User stylesheets)

Autorské styly (Author stylesheets)

Autorské styly s `!important`

Uživatelské styly s `!important`

Specifita (Specificity)/Užší zaměření selektoru:

Čím přesněji selektor prvek popisuje, tím má větší váhu. Specifita se počítá na základě tří složek (ID, Třída, Tag) a inline stylů:

Inline styl (přímo v HTML tagu `style="..."`) → nejvyšší váha (1,0,0,0).

ID selektor (`#menu`) → vysoká váha (0,1,0,0).

Třída, pseudotřída, atribut (`.polozka`, `:hover`, `[type="text"]`) → střední váha (0,0,1,0).

Element/Tag, pseudoelement (`div`, `p`, `::before`) → nejnižší váha (0,0,0,1).

Příklad: Selektor `div.obsah p` (váha: 0,0,1,2) přebije prostý selektor `p` (váha: 0,0,0,1).

Pořadí v kódu (Order of Appearance):

Pokud mají dvě pravidla naprosto stejnou důležitost i specifitu, vyhrává to, které je v CSS souboru napsané později (níže).

C. Blokový model CSS (Box Model)

Blokový model je nejjednodušším základem pro rozvržení prvků (Layout) na stránce. Každý HTML element je prohlížečem interpretován jako obdélníkový box, který se skládá ze čtyř vrstev (od středu ven):

Obsah (Content): Samotný vnitřek elementu, kde se nachází text, obrázek nebo vnořené elementy. Jeho rozměry určují vlastnosti `width` a `height`.

Vnitřní okraj (Padding): Průhledný prostor mezi samotným obsahem a ohraničením prvku. Slouží k tomu, aby se text "nelepil" přímo na okraj. Lze nastavit pro všechny strany (`padding: 10px;`) nebo jednotlivě (`padding-top`, `padding-right`, `padding-bottom`, `padding-left`).

Rámeček / Ohraničení (Border): Čára kolem dokola prvku, která odděluje vnitřní okraj od vnějšího. Může mít definovanou tloušťku (`border-width`), styl (`border-style` : `solid` , `dashed` , `dotted`) a barvu (`border-color`).

Vnější okraj (Margin): Průhledný prostor vně rámečku. Slouží k vytváření rozestupů a mezer mezi jednotlivými nezávislými boxy na stránce. Lze nastavit pro všechny strany (`margin: 10px;`) nebo jednotlivě (`margin-top` , `margin-right` , `margin-bottom` , `margin-left`).

```
.card {
  box-sizing: border-box;
  margin: 1rem;
  padding: 1rem;
  border: 1px solid #ccc;
}
```

Vlastnost `box-sizing` (Kritický chyták u státnic)

`box-sizing: content-box;` **(výchozí chování):** Celková šířka prvku se počítá jako součet šířky obsahu, vnitřního okraje a rámečku.

Celková šířka = `width` + `padding-left` + `padding-right` + `border-left-width` + `border-right-width`

Pokud tedy zadáte prvku `width: 300px; padding: 20px; border: 5px solid black;` , prvek bude na obrazovce reálně široký $300 + (2 \times 20) + (2 \times 5) = 300 + 40 + 10 = 350$ px. To velmi komplikuje přesný návrh designu.

`box-sizing: border-box;` **(moderní řešení):** V tomto režimu je vlastnost `width` rovna celkové výsledné šířce prvku. Vnitřní okraj a rámeček se automaticky "vtlačí" dovnitř a zmenší samotný prostor pro obsah. Ve výše uvedeném případě by prvek měl celkově přesně 300 px a prostor pro obsah by se zmenšil na $300 - (2 \times 20) - (2 \times 5) = 300 - 40 - 10 = 250$ px. V moderním webdesignu se toto nastavení aplikuje globálně na všechny prvky, často pomocí `* { box-sizing: border-box; }` .

D. Typy zobrazení (Display Property)

Vlastnost `display` určuje, jak se element zobrazuje v dokumentu a jak se chová v rámci blokového modelu.

`display: block;`

Element zabírá celou dostupnou šířku a vždy začíná na novém řádku.

Respektuje vlastnosti `width` , `height` , `margin` , `padding` a `border` .

Příklady: `div` , `p` , `h1` , `ul` , `li` .

`display: inline;`

Element zabírá pouze tolik šířky, kolik potřebuje pro svůj obsah.

Nezpůsobuje zalomení řádku (prvky se zobrazují vedle sebe).

Ignoruje nastavení `width` a `height` . Vertikální `margin` a `padding` jsou ignorovány nebo mají omezený efekt.

Příklady: `span` , `a` , `strong` , `em` .

`display: inline-block;`

Kombinuje vlastnosti `inline` a `block` .

Element se zobrazuje vedle sebe s ostatními `inline` nebo `inline-block` prvky (jako `inline`).

Respektuje vlastnosti `width` , `height` , `margin` , `padding` a `border` (jako `block`).

Užitečné pro vytváření navigačních menu nebo prvků, které potřebují mít definovanou šířku a výšku, ale zároveň se mají řadit vedle sebe.

`display: flex;`

Nastaví element jako flex kontejner, což umožňuje flexibilní uspořádání jeho přímých potomků (flex položek) v jednom rozměru (řádek nebo sloupec).

Poskytuje výkonný nástroj pro rozvržení složitých komponent a responzivní design.

Umožňuje snadné zarovnání, rozdělení prostoru a změnu pořadí položek.

E. Praktické použití

Responzivní layout: Přizpůsobení vzhledu stránek různým velikostem obrazovek (mobily, tablety, desktopy).

Typografie: Nastavení fontů, velikostí, barev a zarovnání textu pro zlepšení čitelnosti a estetiky.

Vizuální identita: Aplikace jednotného brandingů a designu napříč celým webem.

Přístupnost webu: Zajištění, že webové stránky jsou použitelné pro co nejširší spektrum uživatelů, včetně těch s postižením.

Oddělení vzhledu od struktury: Udržování HTML čistého a sémantického, zatímco CSS se stará o prezentaci.

F. Nejčastější chyby u zkoušky

Neznalost rozdílu mezi `margin` a `padding`.

Přepisování stylů kvůli specifitě bez pochopení kaskády.

Zapomenutí na `box-sizing` a jeho vliv na výpočet rozměrů prvků.

Záměna `display: block;` a `display: inline;` chování.

Považování CSS za programovací jazyk.

G. Typické zkouškové otázky

Vysvětlete CSS kaskádu a specifitu.

Kaskáda je mechanismus, který určuje, které CSS pravidlo se aplikuje na HTML element, pokud se na něj vztahuje více pravidel pro stejnou vlastnost. Rozhoduje se na základě tří kritérií: původu a důležitosti (např. `!important` přebije běžné styly), specifity selektoru a pořadí v kódu.

Specifita je váha, kterou má selektor. Čím je selektor přesnější (např. `#id` má vyšší specifitu než `.class`, a `.class` má vyšší specifitu než `tag`), tím má vyšší prioritu v kaskádě. Počítá se na základě počtu ID, tříd/pseudotříd/atributů a elementů/pseudoelementů v selektoru. Inline styly mají nejvyšší specifitu.

Popište box model.

Box model popisuje, jak jsou HTML elementy vykreslovány jako obdélníkové boxy. Skládá se ze čtyř vrstev od středu ven:

Content (Obsah): Skutečný obsah elementu (text, obrázky).

Padding (Vnitřní okraj): Průhledný prostor mezi obsahem a rámečkem.

Border (Rámeček): Čára ohraničující padding a content.

Margin (Vnější okraj): Průhledný prostor vně rámečku, který vytváří mezery mezi elementy.

Vlastnost `box-sizing` ovlivňuje, jak se `width` a `height` počítají. `content-box` (výchozí) znamená, že `width` je jen šířka obsahu, zatímco `border-box` znamená, že `width` zahrnuje obsah, padding i border.

Jak funguje `display: block`, `inline` a `flex`?

`display: block;` : Element zabírá celou dostupnou šířku, začíná na novém řádku a respektuje `width`, `height`, `margin` a `padding`. Je to typické chování pro `div`, `p`, `h1`.

`display: inline;` : Element zabírá jen tolik šířky, kolik potřebuje pro svůj obsah, nezpůsobuje zalomení řádku a ignoruje `width` , `height` a vertikální `margin` / `padding` . Je to typické chování pro `span` , `a` , `strong` .

`display: flex;` : Nastaví element jako flex kontejner, což umožňuje jeho přímým potomkům (flex položkám) flexibilní uspořádání v jednom rozměru (řádek nebo sloupec). Poskytuje nástroje pro zarovnání, rozdělení prostoru a změnu pořadí, což je ideální pro responzivní layouty.

H. Shrnutí kapitoly

CSS odděluje vzhled od struktury HTML dokumentu. Kaskáda a specifita určují, které styly se aplikují, zatímco dědění šíří některé vlastnosti hierarchií elementů. Blokový model je základem pro pochopení velikostí a rozvržení prvků na stránce, přičemž `box-sizing` je klíčový pro předvídatelné chování rozměrů. Vlastnost `display` pak definuje základní chování elementu v toku dokumentu, s `flex` jako moderním a výkonným nástrojem pro komplexní rozvržení.

25. Webové aplikace: charakteristika programování na straně serveru a klienta. Základy jazyka JavaScript. DOM a přístup k prvkům stránky.

Klíčové pojmy

webová aplikace

programování na straně klienta (frontend)

programování na straně serveru (backend)

JavaScript

DOM (Document Object Model)

event listener

AJAX

JSON

SPA (Single Page Application)

HTTP API

dynamické typování

prototypová dědičnost

jednovláknové a asynchronní programování

Event Loop

selektor (CSS)

A. Programování na straně serveru (Backend) vs. klienta (Frontend)

Moderní webové aplikace jsou distribuované systémy postavené na architektuře Klient-Server. Vývoj se proto striktně dělí na dvě odlišné domény:

1. Programování na straně klienta (Frontend / Client-side)

Kde to běží: Přimo ve webovém prohlížeči uživatele.

Technologie: HTML (struktura), CSS (vzhled), JavaScript (chování a interaktivita).

Charakteristika: Prohlížeč stáhne ze serveru zdrojové soubory (HTML, CSS, JavaScript) a sám je interpretuje a vykonává. Frontend se stará o uživatelské rozhraní (UI), validaci vstupů ve formulářích před odesláním, animace a dynamické překreslování stránek bez nutnosti znovu načítat celý web ze serveru.

Omezení: Kód je plně viditelný pro uživatele (přes vývojářské nástroje F12) a lze ho na straně klienta modifikovat. Z bezpečnostních důvodů nemá přímý přístup k souborovému systému počítače ani přímo k hlavní databázi aplikace.

2. Programování na straně serveru (Backend / Server-side)

Kde to běží: Na vzdáleném webovém serveru.

Technologie: PHP, Python, Node.js, Java, C#, databáze (SQL, NoSQL).

Charakteristika: Server přijme HTTP požadavek od klienta, skript (např. v PHP) zpracuje data, provede business logiku (např. ověření hesla, komunikaci s databází, generování reportů) a dynamicky vygeneruje výsledný HTML/CSS/JS kód nebo data (např. JSON). Tento vygenerovaný kód nebo data pak pošle zpět klientovi.

Výhody: Kód je před uživatelem skrytý (uživatel vidí až hotový čistý výstup nebo data). Je bezpečný, má plný přístup k databázi, souborům na serveru a výpočetnímu výkonu infrastruktury.

3. Komunikace mezi klientem a serverem

Moderní webové aplikace často komunikují se serverem přes **HTTP API** (Application Programming Interface). Klient posílá HTTP požadavky (např. GET, POST) na server a server odpovídá daty, často ve

formátu **JSON** (JavaScript Object Notation).

AJAX (Asynchronous JavaScript and XML): Umožňuje webové stránce asynchronně komunikovat se serverem na pozadí, aniž by bylo nutné znovu načítat celou stránku. To zlepšuje uživatelský zážitek, protože UI zůstává responzivní. Ačkoliv název obsahuje "XML", dnes se primárně používá JSON pro přenos dat.

4. Single Page Application (SPA)

SPA je webová aplikace, která načte jednu HTML stránku a následné interakce uživatele dynamicky mění obsah stránky pomocí JavaScriptu, aniž by bylo nutné načítat nové stránky ze serveru. Data se získávají asynchronně přes API (pomocí AJAXu). To vede k plynulejšímu uživatelskému zážitku, podobnému desktopovým aplikacím.

B. Základy jazyka JavaScript (JS)

JavaScript je multiparadigmatický (podporuje objektově orientované, imperativní a funkcionální programování), interpretovaný, dynamicky typovaný programovací jazyk, který je nativně podporován všemi moderními webovými prohlížeči.

Hlavní rysy JS:

Dynamické typování: Proměnné se deklarují pomocí klíčových slov `let` (měnitelná proměnná) nebo `const` (konstanta). Datový typ proměnné se neurčuje ručně, ale určuje se automaticky až za běhu podle přiřazené hodnoty. `javascript let vek = 30; // typ number vek = "třicet"; // typ string, typ se změnil za běhu const jmeno = "Alice"; // konstanta typu string`

Prototypová dědičnost: JS nepoužívá klasickou třídní dědičnost (jako Java/C#), ale objekty dědí vlastnosti přímo od jiných objektů (prototypů). Ačkoliv moderní JS syntaxe slovo `class` obsahuje, interně jde stále o syntaktický cukr nad prototypy.

Jednovláknový a asynchronní: JS běží v prohlížeči v jediném hlavním vlákne (Main Thread). Aby dlouhé operace (např. stahování dat ze serveru přes API/AJAX) nezasekly celé uživatelské rozhraní, využívá JS asynchronní programování pomocí:

Callback funkce: Funkce, která je předána jako argument jiné funkci a je zavolána, až když je dokončena nějaká asynchronní operace.

Promises (slibů): Objekty, které reprezentují budoucí výsledek asynchronní operace. Mohou být ve stavu pending, fulfilled nebo rejected.

Konstrukce `async/await`: Syntaktický cukr nad Promises, který umožňuje psát asynchronní kód, jako by byl synchronní, a zlepšuje čitelnost.

Všechny tyto mechanismy jsou postaveny nad mechanismem zvaným **Event Loop**, který neustále kontroluje frontu událostí a callbacků a spouští je, když je hlavní vlákno volné.

C. DOM (Document Object Model) a přístup k prvkům

Když prohlížeč obdrží ze serveru textový HTML dokument, zanalyzuje ho (naparsuje) a vytvoří z něj v paměti objektovou strukturu zvanou DOM (Document Object Model).

Definice DOM:

DOM je stromová struktura objektů, kde každý HTML element, atribut a textový řetězec představuje jeden uzel (Node). Poskytuje programové rozhraní (API) pro HTML a XML dokumenty. Definuje logickou strukturu dokumentů a způsob, jakým je k nim přistupováno a jak s nimi lze manipulovat.

JavaScript funguje jako programový nástroj, který dokáže k tomuto stromu v paměti přistupovat a manipulovat s ním – může elementy mazat, přidávat, měnit jejich CSS styly nebo reagovat na události (kliknutí, stisk klávesy).

Přístup k prvkům stránky (Selekce uzlů)

Abychom mohli s nějakým elementem v JS pracovat, musíme ho nejprve v DOM stromu najít. K tomu slouží metody objektu `document` :

Starší specifické metody:

`document.getElementById('id_prvku')` : Najde prvek podle jeho unikátního ID atributu. Vrátí jeden element nebo `null` .

`document.getElementsByClassName('navez_tridy')` : Vrátí živou kolekci (`HTMLCollection`) všech prvků s danou CSS třídou.

`document.getElementsByTagName('p')` : Vrátí živou kolekci (`HTMLCollection`) všech prvků s daným názvem tagu (např. všechny odstavce).

Moderní univerzální metody (používají standardní CSS selektory):

`document.querySelector('.obsah p')` : Vrátí *první* prvek, který odpovídá zadanému CSS selektoru.

`document.querySelectorAll('div.polozka')` : Vrátí statickou kolekci (`NodeList`) *všech* prvků odpovídajících selektoru.

Manipulace s elementy

Jakmile máme referenci na prvek uloženou v proměnné, můžeme ji měnit:

```
// 1. Výběr prvku
const nadpis = document.querySelector('#hlavni-nadpis');

// 2. Změna textového obsahu uvnitř elementu
nadpis.textContent = 'Nový text nadpisu'; // Mění pouze textový obsah
nadpis.innerHTML = '<em>Nový</em> text nadpisu'; // Mění HTML obsah (včetně tagů)

// 3. Změna CSS stylů (přímý zápis do inline stylů)
nadpis.style.color = 'blue';
nadpis.style.fontSize = '24px';

// 4. Správa CSS tříd (nejlepší praxe pro změnu vzhledu)
nadpis.classList.add('aktivni-styl'); // Přidá CSS třídu
nadpis.classList.remove('neaktivni-styl'); // Odebere CSS třídu
nadpis.classList.toggle('skryty'); // Přepne CSS třídu (přidá/odebere)

// 5. Přidání/odebrání atributů
nadpis.setAttribute('data-info', 'Důležité info');
nadpis.removeAttribute('id');
```

Reakce na události (Event Handling)

JS umožňuje reagovat na akce uživatele (nebo systémové události) pomocí registrace posluchačů událostí (`EventListener`):

```
const tlacitko = document.querySelector('#odesilaci-tlacitko');

// Registrace funkce, která se spustí při kliknutí na tlačítko
tlacitko.addEventListener('click', function(event) {
  alert('Uživatel kliknul na tlačítko!');
  console.log('Událost:', event); // Objekt události obsahuje detaily
});

// Příklad s anonymní arrow funkcí
document.getElementById('input-field').addEventListener('input', (e) => {
```

```
console.log('Změna vstupu:', e.target.value);
});
```

Praktické použití

Interaktivní formuláře (validace, dynamické doplňování)

Mapy a dashboardy (dynamické načítání dat, interaktivní vizualizace)

Klientské validace (okamžitá zpětná vazba uživateli před odesláním formuláře na server)

Komunikace s REST API (načítání a odesílání dat na server bez nutnosti obnovení stránky)

Animace a efekty na webových stránkách

Hry v prohlížeči

Nejčastější chyby u zkoušky

Manipulace s DOM před načtením dokumentu: Pokus o přístup k DOM elementům dříve, než je prohlížeč plně naparsuje (např. spuštění JS kódu v `<head>` bez `defer` nebo `DOMContentLoaded` eventu).

Důvěra v klientskou validaci bez serverové kontroly: Předpoklad, že validace na straně klienta je dostatečná pro bezpečnost a integritu dat. Serverová validace je vždy nezbytná.

Blokování UI dlouhým synchronním kódem: Psaní dlouhých, výpočetně náročných JavaScriptových operací, které běží synchronně v hlavní vlákne, což způsobí zamrznutí uživatelského rozhraní.

Typické zkouškové otázky

Co je DOM? Odpověď: DOM (Document Object Model) je programové rozhraní pro HTML a XML dokumenty. Představuje dokument jako stromovou strukturu uzlů (objektů), kde každý HTML element, atribut a textový řetězec je uzel. Poskytuje způsob, jakým programy (zejména JavaScript) mohou přistupovat ke struktuře, obsahu a stylům dokumentu a dynamicky je měnit.

Jak JavaScript reaguje na události? Odpověď: JavaScript reaguje na události (např. kliknutí myši, stisk klávesy, načtení stránky, odeslání formuláře) pomocí mechanismu zvaného "event handling". Programátor registruje "posluchače událostí" (`event listener`) k určitým DOM elementům. Když dojde k definované události, prohlížeč spustí (zavolá) asociovanou JavaScriptovou funkci (callback), která pak provede požadovanou akci. Příkladem je metoda `addEventListener()` .

Jak webová aplikace komunikuje se serverem? Odpověď: Webová aplikace komunikuje se serverem primárně pomocí protokolu HTTP. Klient (prohlížeč) posílá HTTP požadavky (např. GET pro získání dat, POST pro odeslání dat) na server. Server tyto požadavky zpracuje (např. provede business logiku, komunikuje s databází) a odešle zpět HTTP odpověď, která může obsahovat HTML kód, CSS, JavaScript nebo data ve formátu JSON. Moderní webové aplikace často využívají AJAX pro asynchronní komunikaci s REST API, což umožňuje dynamickou aktualizaci obsahu stránky bez jejího úplného opětovného načtení.

Shrnutí kapitoly

JavaScript doplňuje HTML a CSS o interaktivitu. DOM je rozhraní, přes které program mění obsah a strukturu stránky.

Níže je připravený studijní materiál k okruhu 26, doplněný a naformátovaný dle zadaných pravidel.

26. Jazyk PHP. Problematika uchovávání stavových informací, cookies.

A. Jazyk PHP

Klíčové pojmy:

PHP (Hypertext Preprocessor)

server-side skriptování

dynamicky typovaný jazyk

interpretovaný jazyk

Zend Engine

PHP-FPM

HTML embedding

HTTP odpověď

webový server

PHP (rekurzivní zkratka pro *PHP: Hypertext Preprocessor*) je široce používaný open-source skriptovací jazyk, navržený primárně pro vývoj dynamických webových stránek a webových aplikací. Je dynamicky typovaný a interpretovaný, což znamená, že typy proměnných se určují za běhu a kód se vykonává řádek po řádku (i když moderní PHP používá kompilaci do mezikódu – bytecode – pro Zend Engine, což zvyšuje výkon).

Princip fungování:

Server-side zpracování: PHP kód se spouští na straně serveru. Když klient (webový prohlížeč) požádá o `.php` soubor, webový server (např. Apache, Nginx) předá tento soubor PHP interpretu (často prostřednictvím FastCGI Process Manager – PHP-FPM).

Vkládání do HTML: PHP vzniklo jako jazyk, který lze přímo vkládat do textu HTML dokumentu pomocí speciálních značek `<?php ... ?>`. Interpret zpracuje pouze kód uvnitř těchto značek.

Generování výstupu: PHP kód vykonává operace, jako je interakce s databázemi, zpracování formulářů, generování dynamického obsahu. Výsledky těchto operací (např. data z databáze) se pak vkládají do HTML. Příkazy jako `echo` nebo `print` slouží k odeslání dat do výstupního proudu.

Odeslání klientovi: Po dokončení zpracování PHP interpret vrátí webovému serveru finální HTML (případně jiný typ obsahu, např. JSON). Server pak odešle tuto čistou HTML stránku klientovi v HTTP odpovědi. Z bezpečnostních důvodů a pro oddělení logiky od prezentace klient nikdy nevidí původní PHP kód.

Využití:

PHP je základem mnoha populárních CMS (Content Management System) jako WordPress, Joomla, Drupal a moderních frameworků jako Laravel nebo Symfony, které usnadňují vývoj komplexních webových aplikací.

B. Problematika uchovávání stavových informací

Klíčové pojmy:

bezstavový protokol

HTTP

stavová informace

session management

autentizace

personalizace

sledování uživatelů

Jak bylo zmíněno u otázky č. 23, síťový protokol HTTP je ze své podstaty **bezstavový**. To znamená, že každý HTTP požadavek, který klient odešle serveru, je zpracován zcela izolovaně a server si sám o sobě nepamatuje žádné předchozí interakce s daným klientem. Server neví, zda tentýž uživatel před chvílí navštívil jinou stránku, přihlásil se, nebo si přidal zboží do košíku.

Pro fungování moderních webových aplikací je však nezbytné **stav udržovat**. Bezstavovost HTTP by jinak znemožnila základní funkce jako:

Autentizace uživatele: Po přihlášení by uživatel musel zadávat své údaje při každém dalším požadavku.

Nákupní košík: Obsah košíku by se ztratil po přechodu na jinou stránku.

Personalizace obsahu: Web by nemohl zobrazovat obsah přizpůsobený konkrétnímu uživateli.

Sledování uživatelské aktivity: Analytické nástroje by nemohly sledovat cestu uživatele webem.

K identifikaci uživatele a uchování jeho stavu mezi jednotlivými požadavky se používají dva propojené mechanismy: **Cookies** a **Relace (Sessions)**.

C. Cookies

Klíčové pojmy:

HTTP hlavička `Set-Cookie`

HTTP hlavička `Cookie`

klient-side úložiště

`Max-Age`

`Expires`

`Path`

`Domain`

`HttpOnly`

`Secure`

`SameSite`

XSS (Cross-Site Scripting)

Session Hijacking

velikostní limit

Cookie je malý textový soubor nebo datový záznam (pár klíč=hodnota a volitelná metadata), který server posílá klientovi (webovému prohlížeči) a klient ho ukládá na svém zařízení. Slouží k uchování malého množství dat na straně klienta.

Jak fungují Cookies v HTTP protokolu:

Zápis (server -> klient):

Server v HTTP odpovědi pošle hlavičku `Set-Cookie`.

Příklad: `Set-Cookie: uzivatel=Tomas; Max-Age=3600; Path=/; Domain=example.com; HttpOnly; Secure; SameSite=Lax`.

Prohlížeč tuto informaci přečte a uloží cookie na disk nebo do paměti prohlížeče.

Odesílání (klient -> server):

Při každém dalším HTTP požadavku na danou doménu (a splnění dalších podmínek, jako je `Path` a `Secure`) prohlížeč automaticky přibalí uložené cookie do hlavičky požadavku.

Příklad: `Cookie: uzivatel=Tomas; PHPSESSID=sess_98765fgh`.

Server tak hned při přijetí požadavku ví, o koho jde, nebo může načíst další stavové informace.

Vlastnosti a omezení Cookies:

Uložení na straně klienta: Uživatel je může smazat, zakázat nebo jejich obsah (pokud není šifrovaný) podvrhnout.

Bezpečnostní rizika:

Citlivé údaje: Do cookies se nikdy nesmí ukládat citlivé údaje (jako hesla, bankovní údaje nebo oprávnění), protože jsou snadno zneužitelné.

XSS (Cross-Site Scripting): Pokud webová aplikace obsahuje XSS zranitelnost, útočník může získat přístup k uživatelským cookies a potenciálně převzít jeho relaci.

Session Hijacking: Útočník může ukrást Session ID z cookie a použít ho k předstírání identity legitimního uživatele.

Omezená velikost: Cookies mají omezenou velikost (zpravidla maximálně 4 KB na jednu cookie a omezený počet cookies na doménu).

Metadata (příznaky) pro zvýšení bezpečnosti a kontroly:

`Max-Age` nebo `Expires` : Určuje dobu platnosti cookie. `Max-Age` je preferovaný, udává dobu v sekundách od nastavení. `Expires` udává konkrétní datum a čas.

`Path` : Omezuje, pro které cesty na serveru bude cookie odesílána.

`Domain` : Omezuje, pro které domény bude cookie odesílána.

`HttpOnly` : Zabraňuje JavaScriptu v přístupu k cookie. Chrání před XSS útoky, kdy by útočník mohl skriptem přečíst cookie.

`Secure` : Zajišťuje, že cookie bude odeslána pouze přes šifrované HTTPS spojení.

`SameSite` : Chrání před CSRF (Cross-Site Request Forgery) útoky tím, že omezuje, kdy prohlížeč posílá cookies s cross-site požadavky. Hodnoty mohou být `Lax` , `Strict` nebo `None` .

D. Relace (Sessions) v PHP

Klíčové pojmy:

server-side úložiště

Session ID

```
session_start()
```

```
PHPSESSID
```

```
$_SESSION
```

session fixation

garbage collection

```
session.gc_maxlifetime
```

Protože ukládat veškerá stavová data přímo na straně klienta v cookies je nebezpečné a nepraktické (kvůli velikostním limitům a možnosti manipulace), v praxi se pro uchování stavu používají **Sessions (relace)** spravované přímo na serveru. Cookies se pak používají pouze k uložení **identifikátoru relace (Session ID)**.

Jak funguje Session (mechanismus propojení s Cookie):

Inicializace relace:

Když se na serveru spustí funkce `session_start()` , PHP se pokusí obnovit existující relaci nebo vytvořit novou.

Pokud neexistuje platné Session ID v příchozím požadavku (např. uživatel přichází poprvé), PHP vygeneruje unikátní, náhodný a neuhodnutelný řetězec zvaný Session ID (např. `sess_98765fgh`).

Server-side úložiště:

Server na svém disku (ve výchozím nastavení, ale může být nakonfigurován pro ukládání v databázi, Redis, Memcached apod.) vytvoří soubor s názvem odpovídajícím Session ID.

Do tohoto souboru se pak bezpečně ukládají veškerá data relace pro daného uživatele (např. `přihlasen => true` , `user_id => 42` , `kosik => [...]`). Tato data jsou přístupná v PHP přes superglobální pole `$_SESSION` .

Propojení s klientem (pomocí Cookie):

Server pošle toto Session ID klientovi do prohlížeče jako speciální cookie. Standardně se tato cookie v PHP jmenuje `PHPSESSID` .

Příklad hlavičky `Set-Cookie` : `Set-Cookie: PHPSESSID=sess_98765fgh; Path=/; HttpOnly; Secure` .

Obnovení relace:

Při každém dalším HTTP požadavku prohlížeč automaticky pošle cookie `PHPSESSID` zpět na server.

PHP podle tohoto ID najde příslušný soubor (nebo záznam v databázi) na serveru a obnoví stav aplikace pro daného uživatele, zpřístupní data v `$_SESSION` .

Výhody Sessions:

Bezpečnost: Surová data relace (obsah košíku, stav přihlášení, uživatelská role) jsou uložena na serveru. Uživatel k nim nemá přímý přístup a nemůže je zfalšovat. U sebe v prohlížeči má pouze identifikační klíč (Session ID), který je sám o sobě bezvýznamný bez přístupu k serverovému úložišti.

Kapacita: Na serveru neplatí limit 4 KB jako u cookies, do relace lze uložit velké množství dat.

Životní cyklus: Relace standardně zaniká, jakmile uživatel zavře prohlížeč (pokud není nastaveno jinak pomocí `session.cookie_lifetime` nebo `session.gc_maxlifetime`). Server má také mechanismy pro automatické mazání starých a nepoužívaných relací (garbage collection).

Zabezpečení Sessions:

Regenerace Session ID: Pro ochranu před útoky typu Session Fixation je doporučeno pravidelně regenerovat Session ID (např. po úspěšném přihlášení) pomocí `session_regenerate_id()` .

Použití `HttpOnly` a `Secure` pro `PHPSESSID` cookie: Stejně jako u běžných cookies, i pro `PHPSESSID` je klíčové nastavit tyto příznaky, aby se zabránilo XSS a zajistilo šifrované spojení.

Dostatečná entropie Session ID: Session ID musí být dostatečně dlouhé a náhodné, aby nebylo uhodnutelné.

Typické zkuškové otázky

Jak PHP zpracuje formulář?

Odpověď: PHP zpracuje formulář tak, že po odeslání formuláře (metodou GET nebo POST) server přijme HTTP požadavek. PHP skript, na který formulář odkazuje (definovaný v atributu `action` tagu `<form>`), se spustí. PHP automaticky naplní superglobální pole `$_GET` (pro metodu GET) nebo `$_POST` (pro metodu POST) daty odeslanými z formuláře. Skript pak může přistupovat k těmto datům (např. `$_POST['jmeno']`), validovat je, zpracovat (např. uložit do databáze) a vygenerovat dynamickou HTML odpověď pro klienta.

Jaký je rozdíl mezi cookie a session?

Odpověď:

Cookie: Je malý datový záznam (pár klíč=hodnota) uložený na straně **klienta** (v prohlížeči). Server ji nastaví pomocí hlavičky `Set-Cookie` , a prohlížeč ji automaticky posílá zpět serveru s každým dalším požadavkem na danou doménu. Cookies mají omezenou velikost (cca 4KB) a jsou náchylné k manipulaci uživatelem nebo útokům (XSS).

Session (relace): Je mechanismus pro uchování stavových dat na straně **serveru**. Klient u sebe v cookie (standardně `PHPSESSID`) uchovává pouze unikátní **Session ID**. Server podle tohoto ID najde příslušná data relace (uložená např. v souboru, databázi nebo cache na serveru) a zpřístupní je PHP skriptu (v

`$_SESSION`). Sessions jsou bezpečnější pro ukládání citlivých dat a nemají tak přísné velikostní omezení jako cookies. Cookie se tedy používá k *identifikaci* relace, zatímco relace *uchovává* skutečná data.

Jak zabezpečit autentizační cookie?

Odpověď: Autentizační cookie (která obvykle obsahuje Session ID) lze zabezpečit několika způsoby:

`HttpOnly` **příznak:** Zabraňuje JavaScriptu v přístupu k cookie, čímž chrání před krádeží cookie pomocí XSS útoků.

`Secure` **příznak:** Zajišťuje, že cookie bude odeslána pouze přes šifrované HTTPS spojení, čímž chrání před odposlechem dat během přenosu.

`SameSite` **příznak (např. `Lax` nebo `Strict`):** Chrání před CSRF útoky tím, že omezuje, kdy prohlížeč posílá cookies s cross-site požadavky.

Krátká doba platnosti (`Max-Age`): Omezuje dobu, po kterou je cookie platná, čímž snižuje okno pro potenciální zneužití.

Regenerace Session ID: Pravidelná změna Session ID (např. po přihlášení nebo po určitém čase nečinnosti) pomocí `session_regenerate_id()` pomáhá chránit před Session Fixation útoky.

Ukládání pouze Session ID: Do autentizační cookie by se neměly ukládat žádné citlivé informace přímo, pouze Session ID, které odkazuje na data uložená bezpečně na serveru.

Validace IP adresy/User-Agenta: Server může kontrolovat, zda se IP adresa nebo User-Agent klienta během relace nezměnily, a pokud ano, relaci ukončit.
